INTRODUCTION TO DATA SCIENCE - CASE STUDY REPORT

IPL Score Forecasting & Batting-Collapse Detection using Machine Learning in R

Name	A Badri
Roll No	160124771305
Branch	AI & DS
Section	I1
Dataset	Cricsheet IPL ball-by-ball JSON, 1,169 matches
Language	R (R Markdown)

1. Abstract

This case study builds a data-science pipeline in R on 1,169 official, ball-by-ball IPL matches. Using the runs and wickets of the last 12 balls together with the current match state, it (i) forecasts the final innings score at 6, 10, 15 and 18 overs and (ii) estimates the probability that the batting side will collapse, defined as losing three or more wickets in the next 12 legal balls. Several regression and classification models are compared on matches never seen during training.

2. Introduction and Problem Statement

A run-rate projection (current run rate x 20) is the usual way to estimate a T20 total, but it cannot tell a side that is scoring steadily with wickets in hand from one that has just lost two quick wickets. Collapses are rare but decisive events, so an early warning is valuable to analysts and captains. The project therefore asks two questions:

- What is the expected final score of the innings at a given checkpoint?
- How likely is a batting collapse in the next 12 balls?

3. Objectives

- Parse official ball-by-ball JSON into a clean, analysis-ready table.
- Engineer momentum and pressure features from the last 12 balls.
- Compare baseline, linear, ridge, random-forest and gradient-boosting forecasters.
- Define a collapse mathematically and train classifiers to detect it.
- Provide a reproducible R Markdown report and an interactive what-if simulator.

4. Dataset

Data come from Cricsheet (cricsheet.org), which publishes official ball-by-ball records, one JSON file per match. Each file contains an **info** block (teams, venue, season, overs) and an **innings** list; each innings contains **overs**, and each over contains **deliveries** with *runs* (batter, extras, total), optional *extras* (wides, no-balls) and optional *wickets*.

4.1 Cleaning rules

- Only innings 1 and 2 of full 20-over matches are kept (super overs and rain-shortened games removed).
- Wides and no-balls are folded into the legal ball they accompany so that the ball counter is exact.
- 'Retired hurt' is not counted as a wicket.
- Every legal ball becomes one row with score, wickets, runs and wickets in the last 12 balls, and wickets in the next 12 balls.

[Pending: exact counts after cleaning are written to results/metrics.json when the Rmd is knitted.]

5. Methodology

5.1 Features

Feature	Meaning
cum_score, cum_wickets	Score and wickets at the checkpoint
current_run_rate	Score divided by overs bowled
runs_last12	Runs in the last 12 legal balls (momentum)
wkts_last12	Wickets in the last 12 legal balls (pressure)
checkpoint_overs	6, 10, 15 or 18 overs completed

5.2 Collapse definition

A collapse starts at ball b if the batting side loses **3 or more wickets in the next 12 legal balls**. Inputs come only from balls already bowled, so there is no information leakage. Windows that run past an all-out innings are padded with 'no more wickets'; windows cut short by a successful chase are discarded.

5.3 Models

- **Forecasting:** mean baseline, run-rate x 20, linear regression, ridge regression (glmnet), random forest, gradient boosting (gbm, trees chosen by 5-fold CV).
- **Collapse detection:** logistic regression and random-forest classifier; decision threshold chosen to maximise F1 on training data only.

5.4 Validation

Matches are split 80/20 at random (seed 123) so that no match contributes to both sets. Forecasts are scored with MAE, RMSE and R-squared; collapse classifiers with AUC, precision, recall, F1 and accuracy, since collapses are a minority class.

6. Results

[Pending] Results tables are filled automatically from results/metrics.json after the Rmd is knitted on the full dataset and 'python3 build_docs/build_report.py' is re-run. No figures have been invented for this draft.

[Figure pending: Figure 1: Distribution of first-innings totals - created when IDS_CS.Rmd is knitted on the data]

[Figure pending: Figure 2: Forecast error by checkpoint - created when IDS_CS.Rmd is knitted on the data]

[Figure pending: Figure 3: Random-forest feature importance - created when IDS_CS.Rmd is knitted on the data]

[Figure pending: Figure 4: ROC curves for collapse detection - created when IDS_CS.Rmd is knitted on the data]

[Figure pending: Figure 5: Collapse rate by overs and wickets down - created when IDS_CS.Rmd is knitted on the data]

7. Discussion

The forecasting models are compared against the run-rate x 20 projection at each checkpoint, so the benefit of the last-12-ball features can be read directly from the tables above. Because collapses are a minority class, accuracy alone is not informative; AUC, precision and recall describe how useful the early warning really is.

8. Limitations and Future Work

- No pitch, weather, opposition or batter-quality features.
- Only first-innings totals are forecast because chasing innings end early.
- Random match split; a season-wise split would test drift over time.
- The collapse rule (3 wickets in 12 balls) is a modelling choice; other windows can be tried.
- Future work: batter/bowler form, venue effects, calibrated boosted classifiers, a hosted dashboard.

9. Conclusion

A complete, reproducible R pipeline turns raw official IPL ball-by-ball JSON into two practical outputs: an expected final score and a short-horizon collapse probability, both driven by the runs and wickets of the last 12 balls and the current match state.

10. References

- Cricsheet - ball-by-ball data and JSON format: https://cricsheet.org/downloads/ and https://cricsheet.org/format/json/
- Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.
- Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. Annals of Statistics, 29(5).
- Friedman, Hastie, Tibshirani (2010). Regularization paths for generalized linear models via coordinate descent. J. Stat. Software 33(1).
- R packages: jsonlite, dplyr, ggplot2, caret, randomForest, glmnet, gbm, pROC (CRAN).